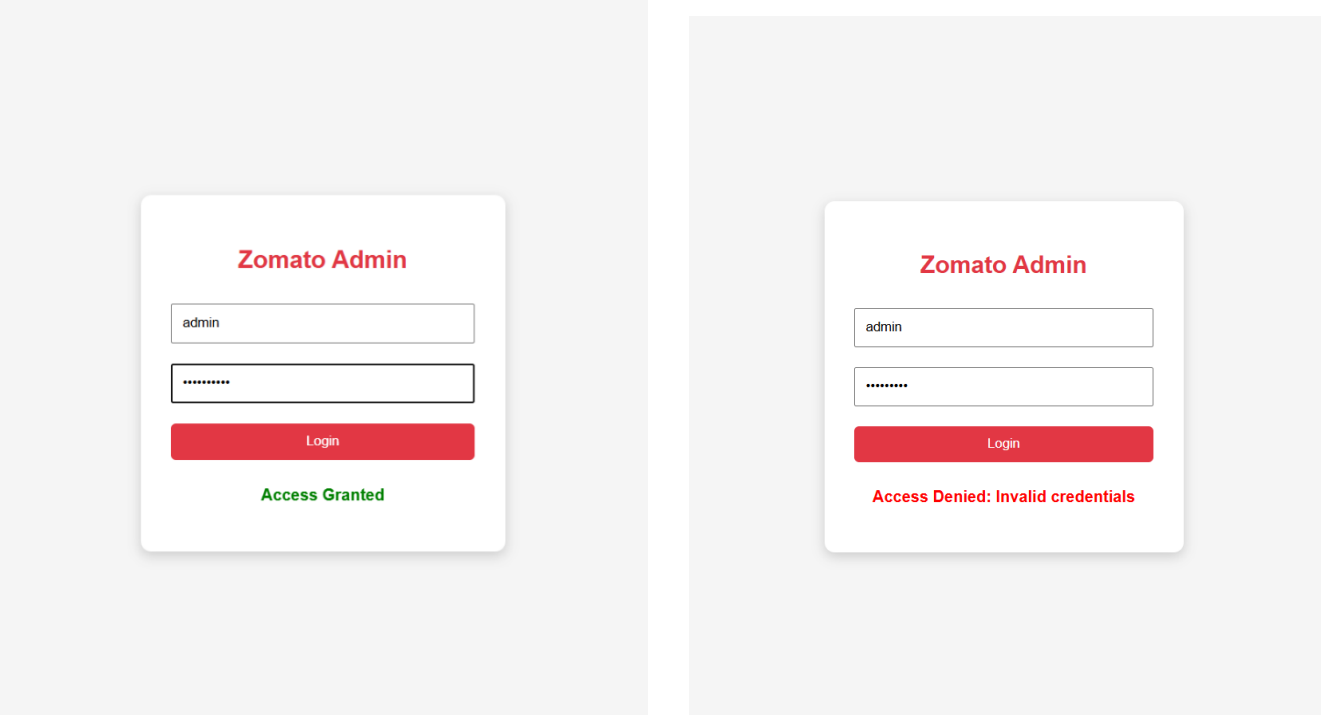
Authentication and Access Control

1.Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.



The mock Zomato Admin Login Form is created using HTML, CSS, and JavaScript. It provides two input fields for the username and password and a Login button to verify the entered credentials.

1. HTML: Creates the login form, including the username field, password field, Login button, and message area.
2. CSS: Styles the form with a white background, red Zomato-themed heading, rounded corners, and a clean layout.
3. JavaScript: Checks the entered username and password against hardcoded admin credentials.
4. Access Granted: If both credentials match, the message “Access Granted” appears in green.
5. Access Denied: If either credential is incorrect, the message “Access Denied: Invalid credentials” appears in red.

Result

The login form successfully verifies the mock admin credentials and displays the appropriate message based on the user's input.

2. Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

This program uses a dictionary to define which resources each role can access. The check_access(user_role, resource) function returns True if access is allowed; otherwise, it returns False.

```
# Define roles and their allowed resources
permissions = {
    "customer": ["order_history"],
    "restaurant_owner": ["menu_edit", "order_history"],
    "admin": ["menu_edit", "order_history", "admin_dashboard"]
}

# Function to check access
def check_access(user_role, resource):
    return resource in permissions.get(user_role, [])

# Test cases
print("Customer - Order History:",
      check_access("customer", "order_history"))

print("Customer - Menu Edit:",
      check_access("customer", "menu_edit"))

print("Restaurant Owner - Menu Edit:",
      check_access("restaurant_owner", "menu_edit"))

print("Restaurant Owner - Admin Dashboard:",
      check_access("restaurant_owner", "admin_dashboard"))
```

```
print("Admin - Admin Dashboard:",
    check_access("admin", "admin_dashboard"))
```

Output

Customer - Order History: True

Customer - Menu Edit: False

Restaurant Owner - Menu Edit: True

Restaurant Owner - Admin Dashboard: False

Admin - Admin Dashboard: True

Explanation

1. Dictionary: The permissions dictionary maps each role to its allowed resources.
2. Function: check_access() checks whether the requested resource exists in the user's permitted resource list.
3. Access control: The function returns True for authorized access and False for unauthorized access.
4. Unknown roles: The .get(user_role, []) method denies access by default if the role is not defined.

3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

This program simulates a Paytm-style two-factor authentication process. First, the user enters a password. If it is correct, the program generates a random 6-digit OTP and displays it in the console. The user must enter the correct OTP to complete the login.

```
import random

# Hardcoded password for demonstration
correct_password = "Paytm@123"

# Step 1: Ask for password
```

```
password = input("Enter your password: ")

if password == correct_password:
    print("Password verified successfully!")

    # Step 2: Generate a 6-digit OTP
    otp = random.randint(100000, 999999)
    print("Your OTP is:", otp)  # Demo only

    # Step 3: Ask the user to enter OTP
    entered_otp = input("Enter the 6-digit OTP: ")

    # Step 4: Verify OTP
    if entered_otp == str(otp):
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed.")

else:
    print("Incorrect password. Login Failed.")
```

output
Enter your password: Paytm@123

Password verified successfully!

Your OTP is: 482615

Enter the 6-digit OTP: 482615

Login Successful
Explanation

Password verification: The program checks whether the entered password matches the stored password.

OTP generation: If the password is correct, random.randint(100000, 999999) generates a 6-digit OTP.

OTP verification: The user enters the OTP, and the program compares it with the generated OTP.

Login result: If both the password and OTP are correct, the program displays Login Successful; otherwise, access is denied.

Conclusion: Two-factor authentication adds an extra layer of security by requiring both a password and a one-time password.

4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

JavaScript Program: Filter Premium Users

This program filters an array of user objects and returns only users who are premium members and have logged in within the last 7 days. As specified, today's date is 2024-06-10.

```
const users = [
  { username: "harsh", isPremium: true, lastLogin: "2024-06-09" },
  { username: "rahul", isPremium: false, lastLogin: "2024-06-08" },
  { username: "priya", isPremium: true, lastLogin: "2024-06-05" },
  { username: "amit", isPremium: true, lastLogin: "2024-06-01" },
  { username: "neha", isPremium: true, lastLogin: "2024-06-10" }
];

function getActivePremiumUsers(users) {
  const today = new Date("2024-06-10T00:00:00Z");
  const sevenDaysAgo = new Date("2024-06-03T00:00:00Z");

  return users.filter(user => {
    const loginDate = new Date(user.lastLogin + "T00:00:00Z");

    return user.isPremium &&
```

```
    loginDate >= sevenDaysAgo &&
    loginDate <= today;
  });
}


console.log(getActivePremiumUsers(users));
```

Output

```
[
  { username: "harsh", isPremium: true, lastLogin: "2024-06-09" },
  { username: "neha", isPremium: true, lastLogin: "2024-06-10" }
]
```

Explanation

1. User array: Stores usernames, premium membership status, and last login dates.
2. Date range: Defines the last seven days, from June 3 to June 10, 2024, inclusive.
3. filter() method: Checks each user against the given conditions.
4. Conditions: A user must have isPremium === true and a lastLogin date within the specified range.
5. Return value: Returns a new array containing only users who satisfy both conditions.

Conclusion: The function uses JavaScript's filter() method to identify premium users who have logged in within the last seven days.

5.Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

No.	Access Control Mistake	Fix / Prevention Tip
1	Broken Access Control: Users can access another user's private data, such as orders or messages.	Verify user permissions and ownership on the server for every request.
2	Excessive Permissions: Regular users can access admin features or perform unauthorized actions.	Apply Role-Based Access Control (RBAC) and follow the principle of least privilege.
3	Insecure Direct Object References (IDOR): Changing an order ID or	Check that the requested resource belongs to the

	profile ID in a URL reveals another user's information.	authenticated user before returning it.
4	Missing API Authorization: Developers protect web pages but forget to secure backend API endpoints.	Enforce authorization checks on every protected API endpoint, not just the frontend.
5	Improper Session Management: Attackers reuse stolen or expired session tokens to access accounts.	Use secure, HTTP-only cookies, expire sessions appropriately, and invalidate tokens after logout.